॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

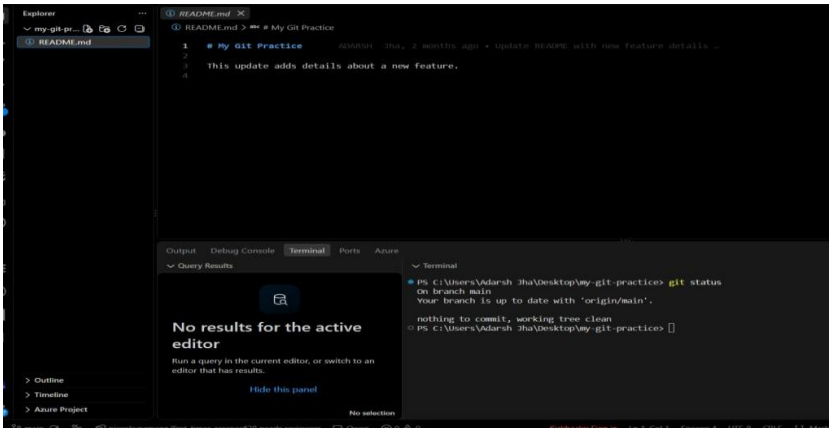
Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

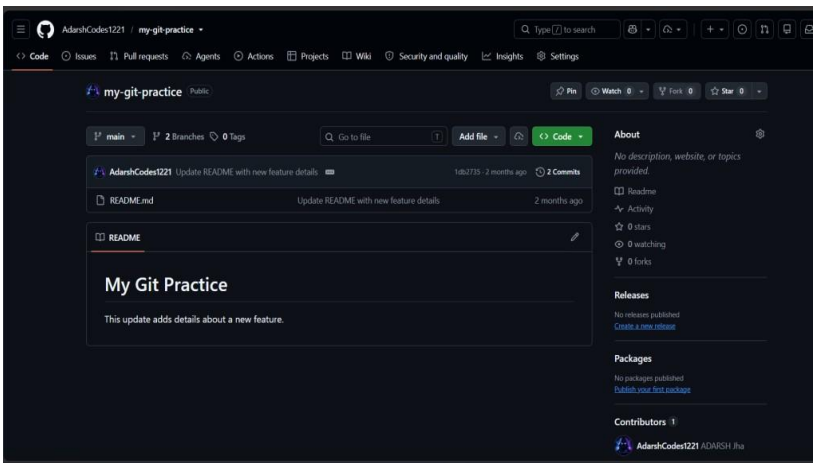
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 01

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Alaa sokhni
PRN No.	23070122276
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	26th October, 2026
Title of Assignment:	Git Setup & Basic Workflow
Theory:	Git is a distributed version control system that helps developers track changes in source code, collaborate efficiently, and maintain different versions of a project. GitHub is a cloud-based platform that hosts Git repositories and enables collaboration. In this assignment, a GitHub repository was created and cloned to the local system. A README.md file was created, staged using git add, committed with git commit, and uploaded to GitHub using git push. This demonstrates the basic Git workflow for managing and synchronizing project files between a local machine and a remote repository. **Commands Used** 1. cd D:\CCodes – Navigated to the directory where the repository would be stored. 2. git clonegit clone https://github.com/AdarshCodes1221/my-git-practice.git– Cloned the remote GitHub repository to the local machine. 3. cd cd my-git-practice – Entered the cloned repository folder. 4. echo "# DevOps Lab" > README.md – Created the README.md file with initial content.

	5. git status – Checked the repository status to verify that README.md was listed as an untracked file. 6. git add README.md – Added the README.md file to the staging area. 7. git status – Verified that the file was staged and ready to be committed. 8. git commit -m "Add initial README.md file" – Created the first commit with a descriptive commit message. 9. git status – Confirmed that the working tree was clean after the commit. 10. git push origin main – Pushed the committed changes from the local repository to the remote GitHub repository.
Implementation:	**Step 1: Create a New Repository on GitHub** Empty Repository on GitHub — Created an empty repository on GitHub without initializing it with a README, .gitignore, or license file to practice the Git local-to-remote workflow. **Step 2: Clone the Repository** Repository Cloned- Cloned the remote GitHub repository to the local machine using git clone and navigated into the project directory. **Step 3: Create README.md** Created README.md- Created the README.md file with initial content and verified using git status that Git recognizes it as an untracked file. **Step 4: Stage the File** Staged README.md- Added the README.md file to the staging area using git add and confirmed it is ready to be committed. **Step 5: Commit the Changes** Committed Changes- Created the first commit with a descriptive commit message and verified that the working tree is clean. **Step 6: Push to GitHub** Pushed to Remote Repository: Uploaded the local commit to the remote GitHub repository using git push. **Step 7: Verify on GitHub** Repository Verified: Verified on GitHub that the README.md file and the latest commit are visible, confirming the push was successful.





Subject Faculty Name: Prof. Arijit Dutta

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

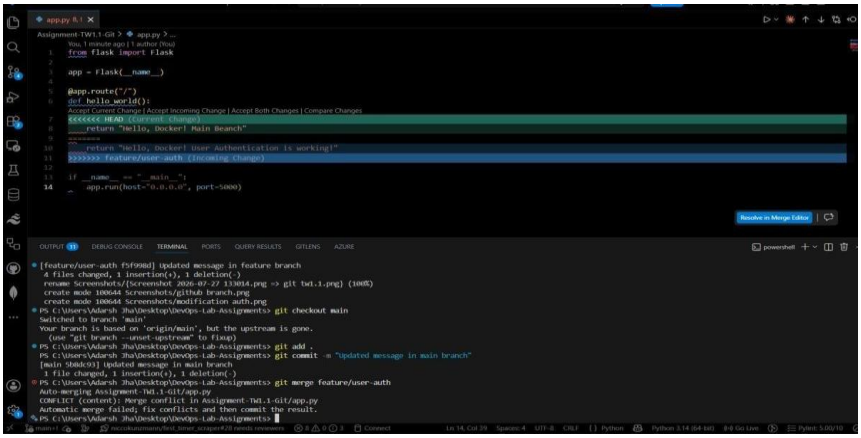
Assignment No. 02

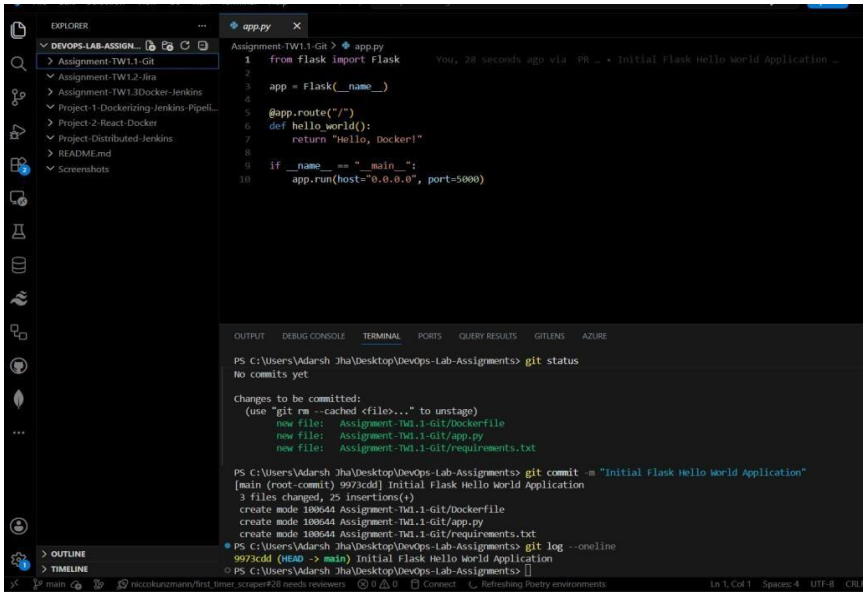
Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Alaa sokhni
PRN No.	23070122276
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	26th October, 2026
Title of Assignment:	Git Branching & Merging
Theory:	Git branching is a feature that allows developers to work on new features or bug fixes independently without affecting the main project. A branch creates a separate line of development where changes can be made safely. Once the work is completed and tested, the branch can be merged back into the main branch using the git merge command. This workflow helps in parallel development, better collaboration, and maintaining a stable main branch. **Commands Used** 1. git status – Checked the current repository status and confirmed that the working tree was clean and the local main branch was up to date. 2. git checkout -b feature-update – Created a new branch named feature-update and switched to it. 3. git branch – Displayed all local branches and verified that feature-update was the active branch. 4. git status – Checked the repository status after modifying the README.md file and confirmed that the changes were detected. 5. git add README.md – Added the modified README.md file to the staging area. 6. git commit -m "Update README on feature branch" – Committed the

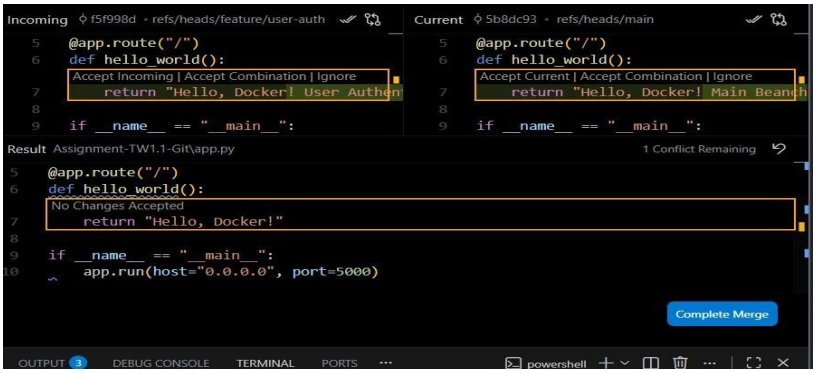
	staged changes to the feature-update branch with a descriptive commit message. 7. git status – Verified that the working tree was clean after committing the changes. 8. git checkout main – Switched back to the main branch. 9. type README.md *(Windows)* – Displayed the contents of the README.md file to verify that the changes from the feature branch were not yet present. 10. git merge feature-update – Merged the feature-update branch into the main branch using a fast-forward merge. 11. type README.md *(Windows)* – Verified that the merged changes were now present in the README.md file. 12. git push origin main – Pushed the updated main branch with the merged changes to the remote GitHub repository.
Implementation:	**Step 1: Verify Current Repository Status** Verified Repository Status — Checked the repository status and confirmed that the working tree was clean and the local main branch was up to date with the remote repository. **Step 2: Create and Switch to a New Branch** Created Feature Branch — Created a new branch named feature-update and switched to it successfully using git checkout -b. **Step 3: Modify README.md** Modified README.md — Updated the README.md file on the feature-update branch and verified that Git detected the modified file. **Step 4: Stage and Commit the Changes** Committed Feature Changes — Staged and committed the updated README.md file on the feature-update branch with a descriptive commit message. **Step 5: Switch Back to Main Branch** Switched to Main Branch — Switched back to the main branch and verified that the feature branch changes were not yet available. **Step 6: Merge the Feature Branch**Merged Feature Branch — Successfully merged the feature-update branch into the main branch using a fast-forward merge. **Step 7: Push Updated Main Branch**Pushed Updated Main Branch — Uploaded the merged changes from the local main branch to the remote GitHub repository.

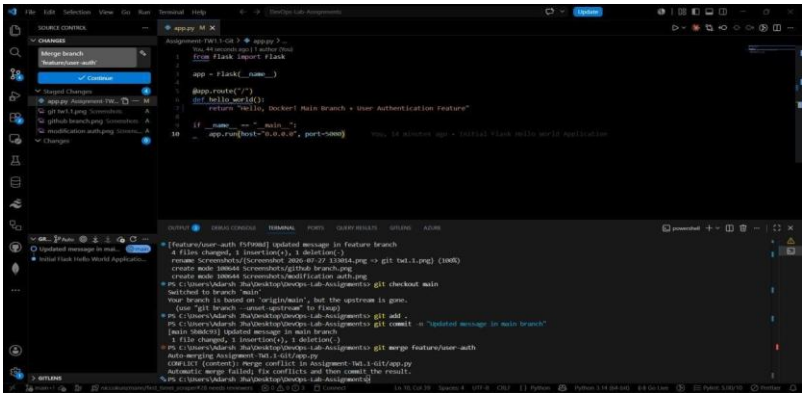
Step 8: Verify on GitHub

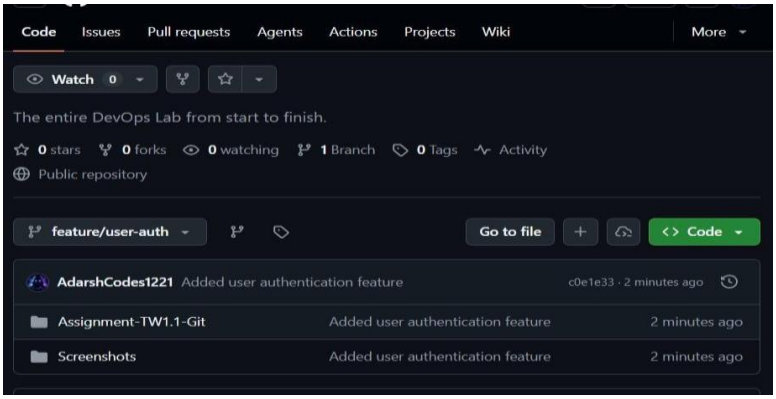
Repository Verified — Verified on GitHub that the merged changes are present in the main branch and that the commit history reflects both commits.











Subject Faculty Name: Prof. Arijit Dutta

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 03

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Alaa sokhni
PRN No.	23070122276
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	26th October, 2026
Title of Assignment:	Simulating Git Collaboration & Conflict Resolution
Theory:	Git collaboration allows multiple developers to work on the same project simultaneously using separate branches. When different branches modify the same part of a file, Git cannot automatically combine the changes and reports a merge conflict. The conflict must be resolved manually by editing the affected file, removing the conflict markers, and selecting the final version of the content. After resolving the conflict, the changes are staged, committed, and pushed to the remote repository. This process ensures that all changes are correctly integrated while maintaining the project's history. **Commands Used** 1. git checkout main – Switched to the main branch before starting the experiment. 2. git status – Verified that the working tree was clean and the main branch was up to date. 3. git checkout -b conflicting-feature – Created a new branch named conflicting-feature and switched to it. 4. Modified the README.md file and changed the first line on the conflicting-feature branch. 5. git add README.md – Staged the modified README.md file on the feature branch.

	6. git commit -m "Update README first line on feature branch" – Committed the changes made on the conflicting-feature branch. 7. git checkout main – Switched back to the main branch. 8. Modified the same first line of the README.md file differently on the main branch. 9. git add README.md – Staged the modified file on the main branch. 10. git commit -m "Update README first line on main" – Committed the changes on the main branch. 11. git merge conflicting-feature – Attempted to merge the conflicting-feature branch into the main branch, resulting in a merge conflict. 12. git status – Verified the conflict by checking that README.md appeared under Unmerged paths. 13. Manually edited the README.md file to remove the conflict markers and resolve the conflict. 14. git add README.md – Staged the resolved file after resolving the merge conflict. 15. git status – Confirmed that all conflicts were fixed and the repository was ready for the merge commit. 16. git commit -m "Merge conflicting-feature, resolve README conflict" – Completed the merge by creating a merge commit after resolving the conflict. 17. git push origin main – Pushed the resolved merge commit from the local main branch to the remote GitHub repository.
Implementation:	**Step 1: Verify Current Repository Status** Verified Repository Status — Confirmed that the repository was on the main branch with a clean working tree before creating the conflicting branch. **Step 2: Create and Switch to a New Branch** Created Conflicting Branch — Created a new branch named conflicting-feature from the current state of the main branch. **Step 3: Modify README.md on the Feature Branch** Updated Feature Branch — Modified the first line of README.md on the conflicting-feature branch and committed the changes. **Step 4: Switch Back to Main and Make a Different Change** Updated Main Branch — Switched back to the main branch, modified the same line differently, and committed the changes. **Step 5: Merge the Branch** Merge Conflict — Attempted to merge the conflicting-feature branch into main, resulting in a merge conflict because both branches modified the same line. **Step 6: Verify the Conflict**

Conflict Verified — Confirmed the merge conflict using git status, which displayed README.md under Unmerged paths.

Step 7: Resolve the Conflict

Conflict Resolved — Manually removed the conflict markers and edited the file to create the final merged version.

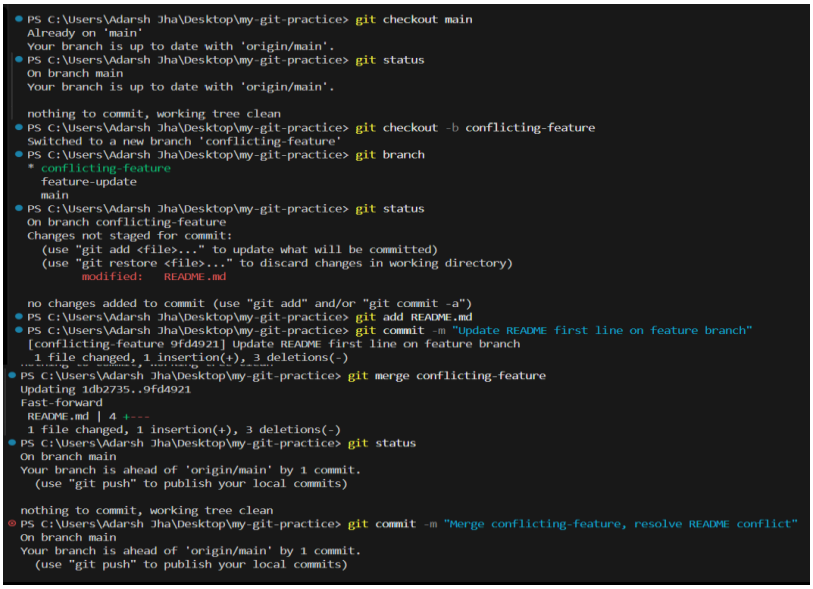
Step 8: Stage the Resolved File

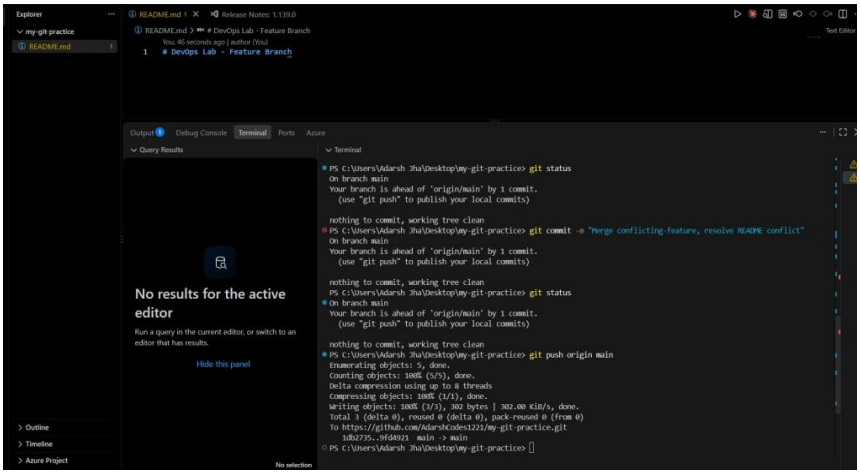
Resolved File Staged — Staged the resolved file and confirmed that all merge conflicts had been fixed.

Step 9: Complete the Merge Merge Completed — Completed the merge by creating a merge commit that combines the histories of the main and conflicting-feature branches after resolving the conflict.

Step 10: Push to GitHub

Pushed to GitHub — Uploaded the resolved merge commit to the remote GitHub repository





Subject Faculty Name: Prof. Arijit Dutta

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 04

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Alaa sokhni
PRN No.	23070122276
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	26th October, 2026
Title of Assignment:	Jira Project & Issue Tracking
Theory:	Jira is a project management and issue tracking tool developed by Atlassian that helps teams plan, organize, assign, and monitor project tasks efficiently. It supports Agile methodologies such as Scrum and Kanban, allowing teams to manage work through customizable workflows. In this experiment, a Team-managed Kanban project was created to simulate a software development workflow. Three different issue types—Story, Task, and Bug—were created to represent a feature request, a development task, and a software defect. These issues were tracked visually on the Kanban board by moving them through the workflow stages To Do, In Progress, and Done. This experiment demonstrates how Jira helps in organizing project activities, tracking task progress, managing software issues, and improving collaboration among team members using an efficient issue-tracking system.
Implementation:	**Step 1: Create a New Jira Project**

Created Kanban Project — Successfully created a Team-managed Kanban project named Hello Docker and verified the Kanban board with the default workflow columns (To Do, In Progress, and Done).

Step 2: Create a Story Issue

Created Story Issue — Created a Story issue named Develop login page with the description *"User should be able to log in using email and password"* to represent a new feature in the project.

Step 3: Create the Task Issue

Created Task Issue — Created a Task issue named Set up database with the description *"Create and configure the project database"* to represent a development task in the project.

Step 4: Create the Bug Issue

Created Bug Issue — Created a Bug issue named Submit button not working with the description *"Clicking the submit button on the contact form does nothing"* to report a defect in the application.

Step 5: Verify All Issues on the Board

Initial Board View — Verified that the Story, Task, and Bug issues were successfully created and appeared in the To Do column of the Kanban board.

Step 6: Move the Task

Task Completed — Moved the Set up database task from To Do to Done, indicating successful completion of the task.

Step 7: Move the Story

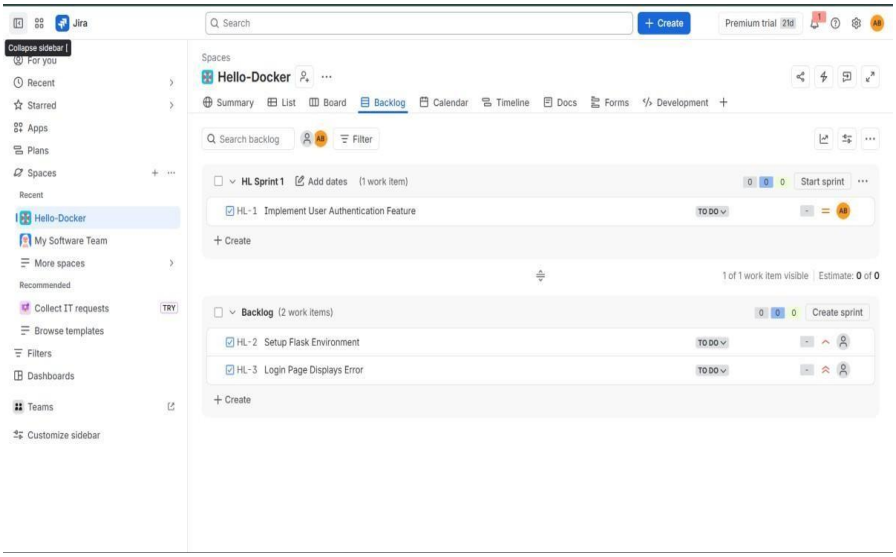
Story Completed — Moved the Develop login page Story issue from To Do to Done, indicating successful completion of the feature implementation.

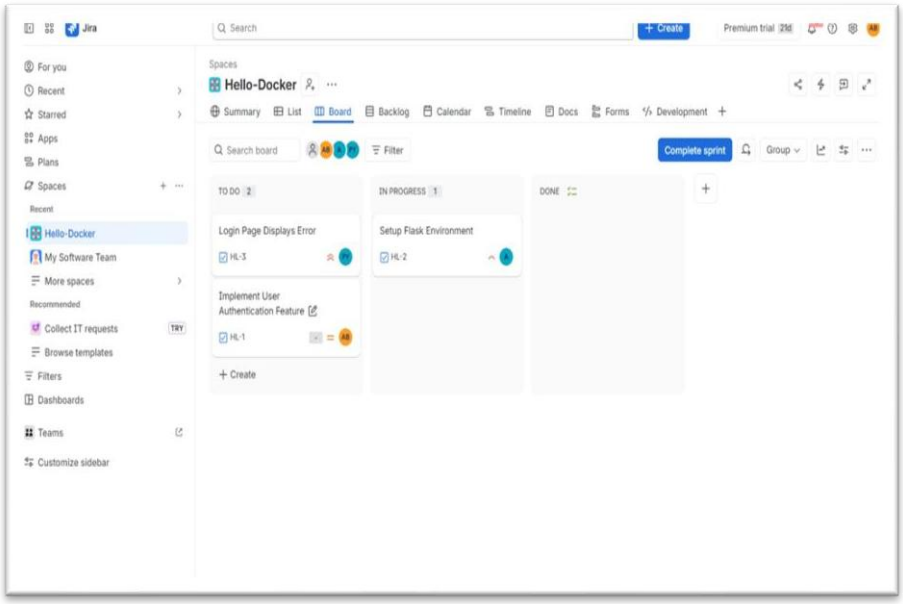
Step 8: Move the Bug

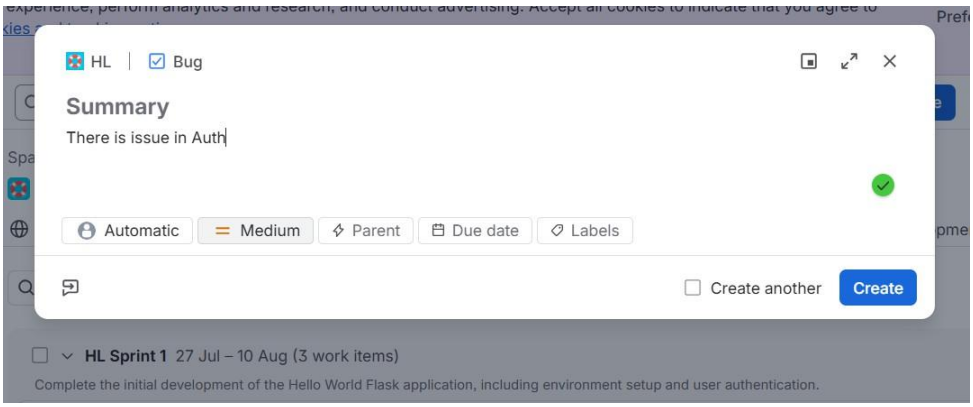
Bug Resolved — Moved the Submit button not working Bug issue from To Do to Done, indicating that the reported defect was successfully resolved.

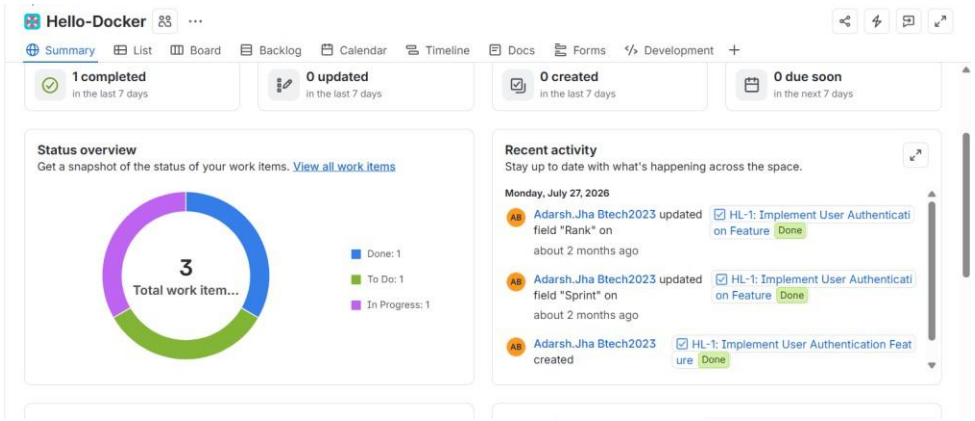
Step 9: Final Verification

Project Verification — Verified that all issues were successfully created and progressed through the Kanban workflow from To Do to Done, demonstrating effective issue tracking and project management in Jira.


Jira
Search
Create
Premium trial 21d
Collapse sidebar
For you
Recent
Starred
Apps
Plans
Spaces
Recent
Hello-Docker
My Software Team
More spaces
Recommended
Collect IT requests
TRY
Browse templates
Filters
Dashboards
Teams
Customize sidebar
Spaces
Hello-Docker
Summary
List
Board
Backlog
Calendar
Timeline
Docs
Forms
Development
Search backlog
Filter
HL Sprint 1
Add dates
(1 work item)
Start sprint
HL-1 Implement User Authentication Feature
TO DO
Create
1 of 1 work item visible
Estimate: 0 of 0
Backlog (2 work items)
Create sprint
HL-2 Setup Flask Environment
HL-3 Login Page Displays Error


Jira
Search
Create
Premium trial 21d
For you
Recent
Starred
Apps
Plans
Spaces
Recent
Hello-Docker
My Software Team
More spaces
Recommended
Collect IT requests
TRY
Browse templates
Filters
Dashboards
Teams
Customize sidebar
Spaces
Hello-Docker
Summary
List
Board
Backlog
Calendar
Timeline
Docs
Forms
Development
Search board
Filter
Complete sprint
Group
TO DO 2
IN PROGRESS 1
DONE
Login Page Displays Error
HL-3
Setup Flask Environment
HL-2
Implement User Authentication Feature
HL-1
Create


HL
Bug
Summary
There is issue in Auth
Automatic
Medium
Parent
Due date
Labels
Create another
Create
HL Sprint 1 27 Jul – 10 Aug (3 work items)
Complete the initial development of the Hello World Flask application, including environment setup and user authentication.



Subject Faculty Name: Prof. Arijit Dutta

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 05

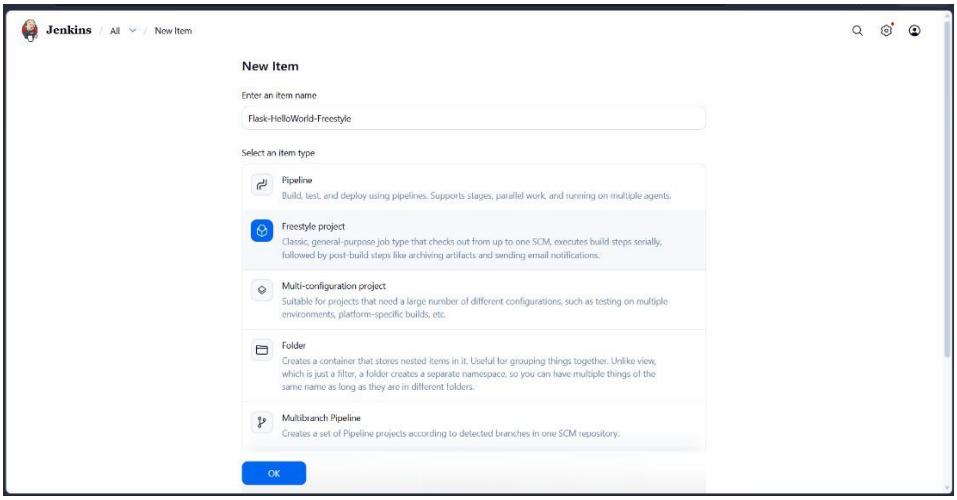
Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Alaa sokhni
PRN No.	23070122276
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	26th October, 2026
Title of Assignment:	Continuous Integration with Jenkins
Theory:	**Theory** Jenkins is an open-source automation server used to implement Continuous Integration (CI) and Continuous Delivery (CD). It automates software development tasks such as building, testing, and deploying applications. Jenkins integrates with version control systems like Git to automatically retrieve source code and execute predefined build steps. In this experiment, Jenkins was installed using Docker, configured with the Git plugin, and connected to a GitHub repository. A Freestyle project was created to pull source code from the repository and execute shell commands. The build was then triggered manually, and the console output was examined to verify successful execution. **Commands Used** • docker --version – Verified that Docker was installed on the system. • docker ps – Checked whether the Jenkins Docker container was running. • docker run -d -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home --name myjenkins jenkins/jenkins:lts-jdk17 – Created and started the Jenkins Docker container. • docker logs myjenkins – Displayed the Jenkins logs to obtain the initial administrator password during the initial setup.

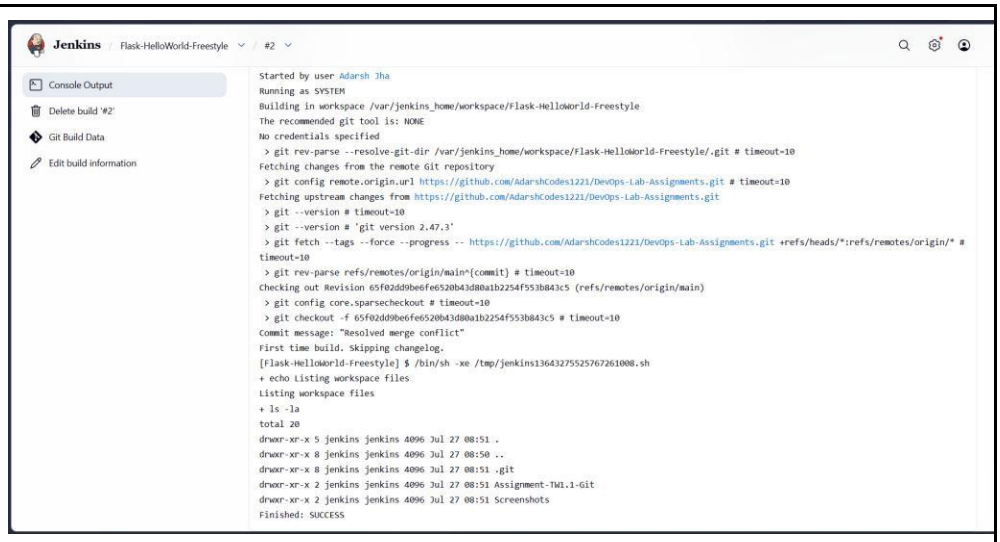
	• docker exec myjenkins cat /var/jenkins_home/secrets/initialAdminPassword – Retrieved the initial administrator password generated by Jenkins. **Execute Shell Commands** • echo "BUILDING: ${JOB_NAME}" – Displayed the Jenkins job name. • echo "BUILD NUMBER: ${BUILD_NUMBER}" – Displayed the current build number. • echo "WORKSPACE: ${WORKSPACE}" – Displayed the Jenkins workspace path. • ls -la – Listed all files present in the Jenkins workspace. • echo "Hello Jenkins!" – Displayed a sample message to verify successful execution of the build step.
Implementation:	**Step 1: Verify Docker and Jenkins Container** Verified Docker and Jenkins Container — Verified Docker installation and confirmed that the Jenkins Docker container (myjenkins) was running successfully. **Step 2: Access Jenkins Dashboard** Accessed Jenkins Dashboard — Successfully accessed the Jenkins Dashboard and verified that the existing Freestyle project (Git-Practice-Build) was available. **Step 3: Verify Git Plugin Installation** Verified Git Plugin — Verified that the Git plugin was installed and enabled, allowing Jenkins to integrate with Git repositories for source code management. **Step 4: Open the Freestyle Project** Opened Freestyle Project — Opened the existing Freestyle project (Git-Practice-Build) to verify its configuration and prepare it for executing the build process. **Step 5: Verify Source Code Management (Git)** Configured Source Code Management — Configured the Freestyle project to retrieve source code from the GitHub repository by selecting Git as the Source Code Management option and specifying the repository URL. **Step 6: Verify Build Step** Configured Build Step — Verified the Execute shell build step containing shell commands to display build information, list the repository files, and print a sample message during the build process. **Step 7: Save and Run the Build** Executed Build — Successfully executed the Freestyle project using Build Now and verified that the build completed successfully with a SUCCESS status.

Step 8: View Console Output

Verified Console Output — Examined the console output and verified that Jenkins successfully cloned the Git repository, executed the configured shell commands, and completed the build with a SUCCESS status.







Subject Faculty Name: Prof. Arijit Dutta

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 06

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Alaa sokhni
PRN No.	23070122276
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	26th October, 2026
Title of Assignment:	Basic Declarative Pipeline
Theory:	**Theory** A Jenkins Pipeline is a suite of plugins that enables Continuous Integration and Continuous Delivery (CI/CD) by defining the entire software development workflow as code. It allows developers to automate tasks such as source code checkout, building, testing, and deployment. A Declarative Pipeline provides a structured and simplified syntax for defining pipeline stages using Groovy. It consists of key components such as pipeline, agent, stages, stage, and steps. The agent specifies where the pipeline will run, stages divide the workflow into logical phases, and steps define the actions to be executed within each stage. In this experiment, a Declarative Pipeline project was created in Jenkins. The pipeline checked out source code from a GitHub repository, executed shell commands to verify the repository contents, and displayed the build output. The successful execution of each stage was verified through the Stage View and Console Output. **Commands Used** **Pipeline Script Components** • pipeline {} – Defines the complete Declarative Pipeline. • agent any – Specifies that the pipeline can run on any available Jenkins

	agent. • stages {} – Groups all stages of the pipeline. • stage('Checkout Source Code') – Defines the stage for retrieving source code from GitHub. git url: 'https://github.com/AdarshCodes1221/my-git-practice.git', branch: 'main' – Checks out the source code from the specified GitHub repository. • stage('Run Example Command') – Defines the stage for executing shell commands. • echo 'message' – Displays informational messages in the Jenkins build log. • sh 'echo "Hello from Declarative Pipeline!"' – Executes a shell command to print a sample message. • sh 'cat README.md' – Displays the contents of the README.md file from the repository. • post { always { ... } } – Executes the specified actions after the pipeline finishes, regardless of the build result.

	Step 1: Create a New Pipeline Project
	
	Created Pipeline Project — Created a new Jenkins Pipeline project named Declarative-Pipeline-Demo by selecting the Pipeline project type.
	
	Step 2: Define the Pipeline Script
	
	pipeline {
	
	agent any
	
	parameters {
	
	string(
	
	name: 'GREETING_NAME',
	
	defaultValue: 'Adarsh',
	
	description: 'The name to use in the greeting'
	
	)
	
	}
	
	stages {

```
stage('Greeting') {
    steps {
        echo "Hello ${params.GREETING_NAME}"
    }
}

stage('Checkout Source Code') {
    steps {
        echo 'Starting Git checkout...'

        git url: 'https://github.com/AdarshCodes1221/my-git-practice.git',
            branch: 'main'

        echo 'Git checkout completed.'
    }
}

stage('Run Example Command') {
    steps {
        echo 'Executing shell commands...'

        sh 'echo "Hello from Declarative Pipeline!"'
```

```
            sh 'echo "--- Reading README ---"'


            sh 'cat README.md'
        }
    }
}


post {
    always {
        echo 'Pipeline finished.'
    }
}
```

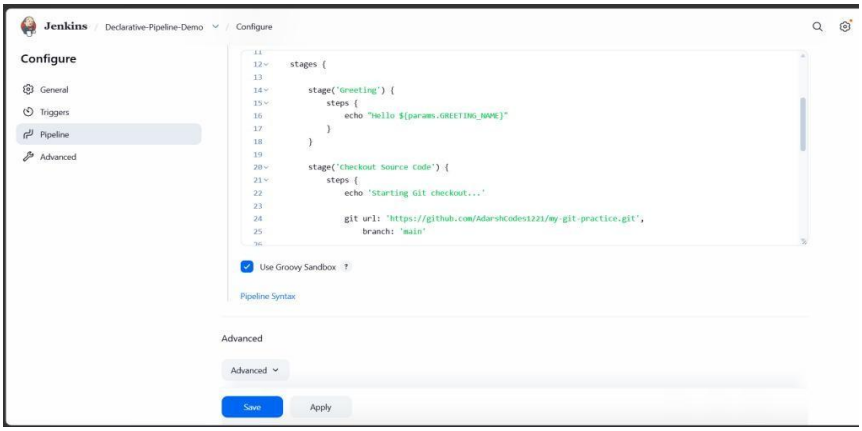
Defined Declarative Pipeline Script — Configured the Jenkins Pipeline using a Declarative Pipeline script to check out source code from the GitHub repository and execute shell commands in separate stages.

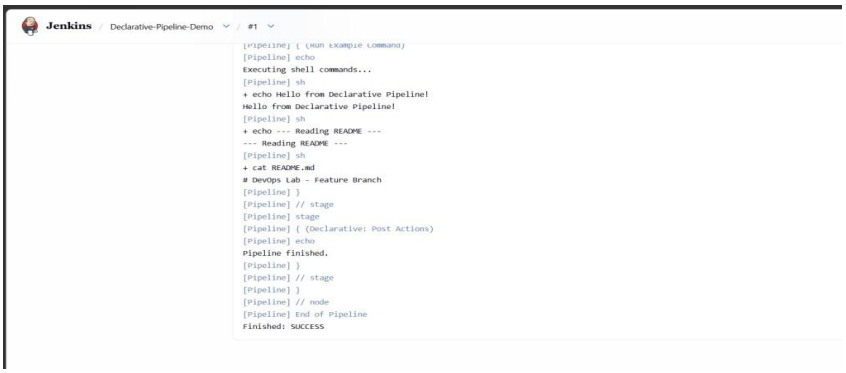
Step 3: Save and Run the Pipeline

Executed Declarative Pipeline — Successfully executed the Declarative Pipeline and verified that all stages, including Checkout Source Code and Run Example Command, completed successfully.

Step 4: View Console Output

Verified Console Output — Examined the console output and verified that Jenkins successfully checked out the Git repository, executed the pipeline stages, ran the shell commands, and completed the pipeline with a SUCCESS status.





Subject Faculty Name: Prof. Arijit Dutta

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 07

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Alaa sokhni
PRN No.	23070122276
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	26th October, 2026
Title of Assignment:	Pipeline with Parameters
Theory:	**Declarative Pipeline with Parameters** A Declarative Pipeline is a Jenkins feature that allows the build, test, and deployment process to be defined as code using a structured Groovy syntax. It provides a simple and readable way to automate software development workflows. Pipeline parameters enable users to provide input values at the time of execution, making the pipeline flexible and reusable. In this experiment, a string parameter (GREETING_NAME) was added to the pipeline, allowing the user to enter a custom value during execution. The pipeline accessed this value using the params object and displayed it in the console output. This demonstrates how parameterized pipelines can accept user input and perform dynamic operations based on the provided values.
Implementation:	**Step 1: Open the Existing Pipeline Project** Opened Pipeline Configuration — Opened the configuration page of the existing Declarative-Pipeline-Demo project to modify the pipeline script and add build parameters. **Step 2: Add the Parameters Block & Save**

Modified Script:

Configured Parameterized Pipeline — Added the GREETING_NAME string parameter to the Declarative Pipeline, updated the pipeline script to use the parameter in the greeting message, and saved the updated pipeline configuration.

Step 3: Build with Parameters

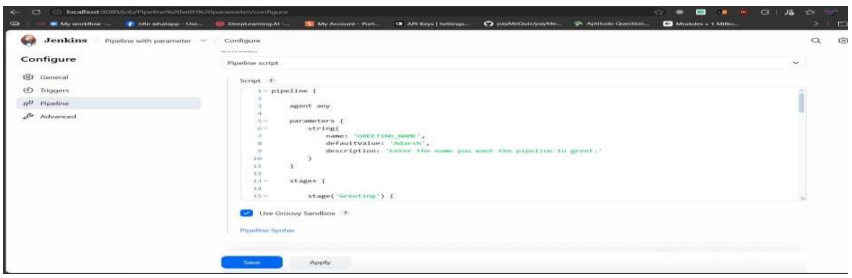
Provided Build Parameter — Used the Build with Parameters option and entered the value for the GREETING_NAME parameter before executing the Declarative Pipeline.

Step 4: Execute the Parameterized Pipeline

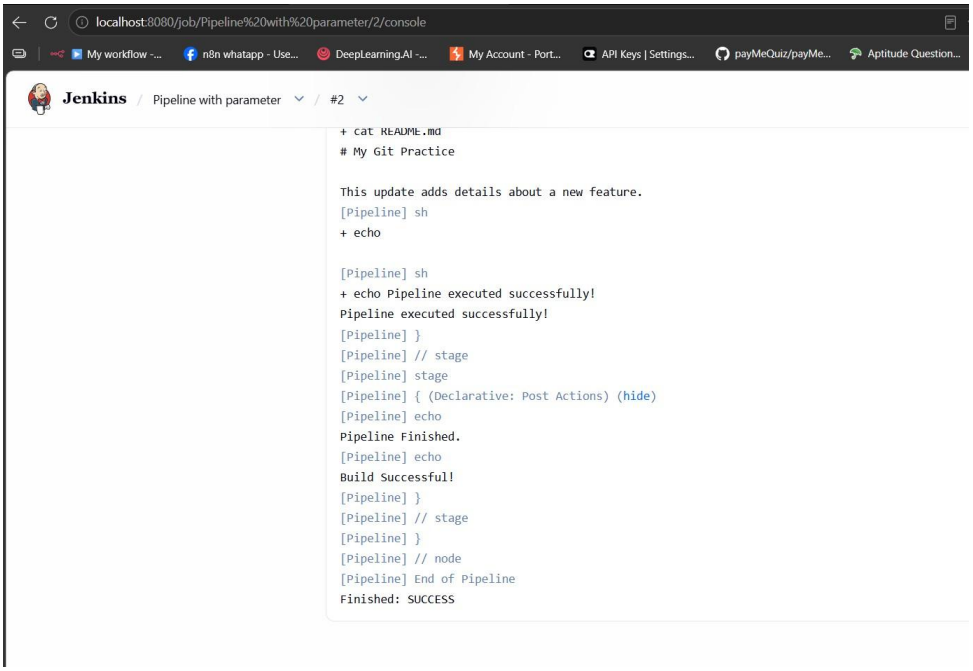
Executed Parameterized Pipeline — Successfully executed the parameterized Declarative Pipeline using the provided GREETING_NAME value and verified that the build completed successfully.

Step 5: Verify the Parameter Usage

Verified Parameter Usage — Examined the console output and verified that the pipeline successfully used the GREETING_NAME parameter value (World) during execution. The pipeline completed successfully with a Finished: SUCCESS status.







Subject Faculty Name: Prof. Arijit Dutta

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

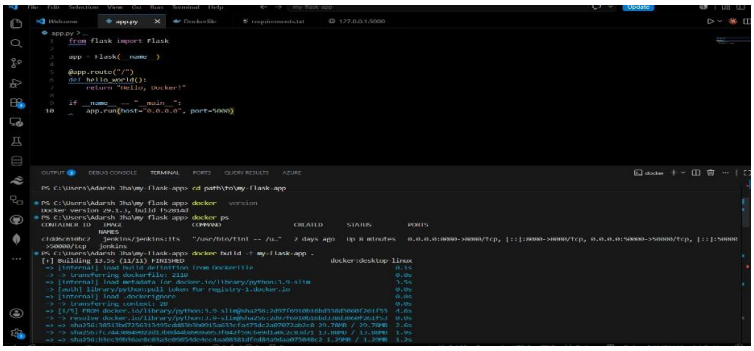
(Established under section 3 of the UGC Act, 1956)

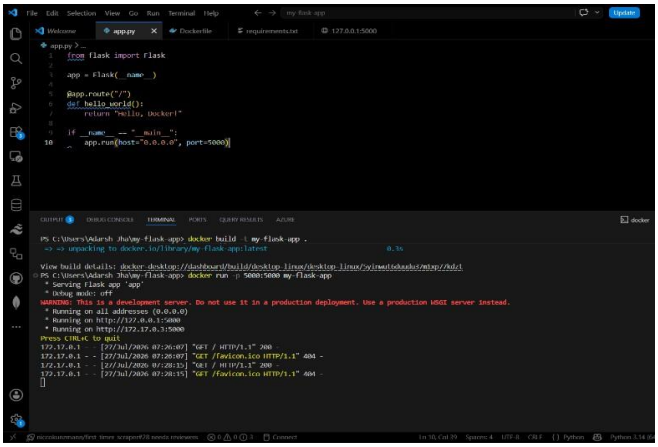
Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

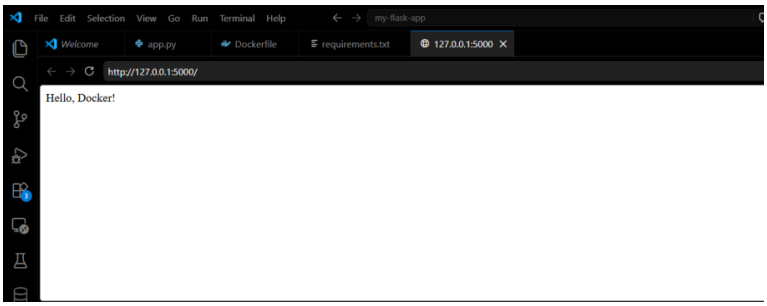
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 08

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Alaa sokhni
PRN No.	23070122276
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	26th October, 2026
Title of Assignment:	Dockerfile & Image Build
Theory:	Docker is a containerization platform that enables applications and their dependencies to be packaged into lightweight, portable containers. A Dockerfile contains the instructions required to build a Docker image, while a Docker image serves as a template for creating containers. A Docker container is a running instance of an image that provides an isolated environment for executing applications consistently across different systems. In this experiment, a simple Flask web application was containerized by creating the required application files, building a Docker image, running the container, and verifying the application through a web browser. **Commands Used** • mkdir my-flask-app – Created a new directory for the Flask application. • cd my-flask-app – Navigated to the project directory. • docker build -t my-flask-app . – Built a Docker image named my-flask-app from the Dockerfile. • docker build --no-cache -t my-flask-app . – Rebuilt the Docker image without using cached layers after updating the dependencies. • docker run -p 5000:5000 my-flask-app – Started the Docker container and mapped port 5000 of the container to port 5000 on the host machine for browser access.

Implementation:	**Step 1: Create the Flask Application** Created Flask Application — Created the my-flask-app project directory and added the required application files. Implemented the app.py file containing a simple Flask web application that displays the message "Hello, Docker!" when accessed. **Step 2: Create the Requirements File** Created Requirements File — Created the requirements.txt file specifying the Flask dependency required to install the necessary packages for the Flask application. **Step 3 – Create the Dockerfile** Created Dockerfile — Created the Dockerfile specifying the Python base image, working directory, application dependencies, exposed port, and startup command required to containerize the Flask application. **Step 4 – Build the Docker Image** Built Docker Image — Successfully built the Docker image named my-flask-app using the Dockerfile. Docker completed all build stages and created the container image successfully. **Step 5 – Run the Docker Container** Executed Docker Container — Successfully started the Docker container and launched the Flask application. The application is running on port 5000 and is accessible through the host machine. **Step 6: Verify the Flask Application** Verified Dockerized Flask Application — Opened the Flask application in a web browser using http://localhost:5000 and successfully verified that the Docker container served the "Hello, Docker!" webpage. 


app.py
from flask import Flask
app = Flask(__name__)
@app.route("/")
def hello_world():
return "Hello, Docker!"
if __name__ == "__main__":
app.run(host="0.0.0.0", port=5000)
PS C:\Users\Adarsh Jha\my-flask-app> docker build -t my-flask-app .
PS C:\Users\Adarsh Jha\my-flask-app> docker run -p 5000:5000 my-flask-app
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.3:5000
Press CTRL+C to quit
172.17.0.1 - - [27/Jul/2026 07:26:07] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [27/Jul/2026 07:26:07] "GET /favicon.ico HTTP/1.1" 404 -
172.17.0.1 - - [27/Jul/2026 07:28:15] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [27/Jul/2026 07:28:15] "GET /favicon.ico HTTP/1.1" 404 -


http://127.0.0.1:5000/
Hello, Docker!

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

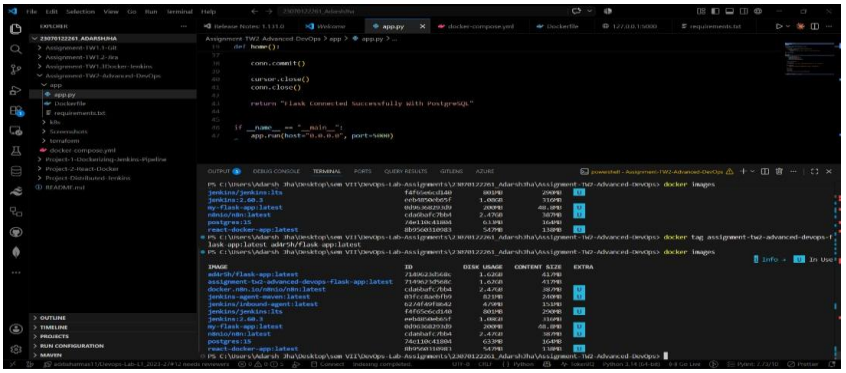
Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

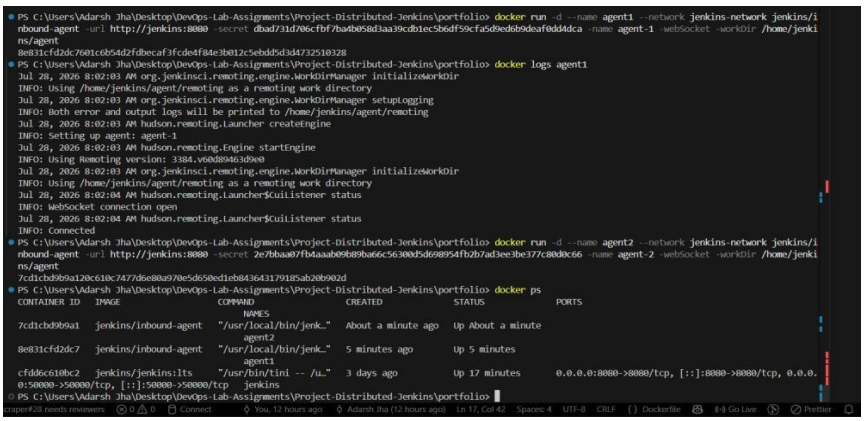
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 09

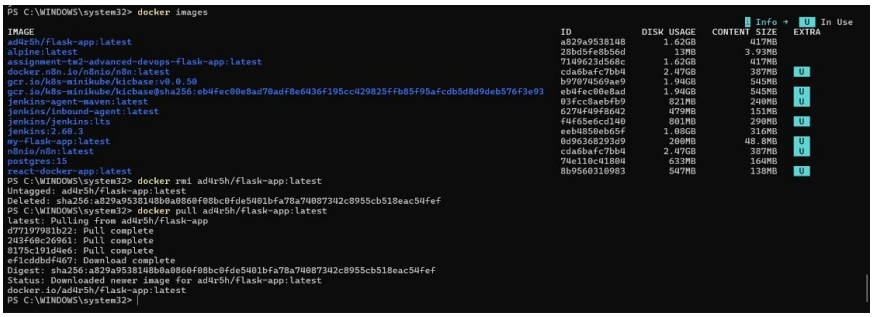
Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Alaa sokhni
PRN No.	23070122276
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	26th October, 2026
Title of Assignment:	Container Management & Networking
Theory:	Docker container management is the process of creating, running, monitoring, stopping, inspecting, and removing containers. Port mapping allows applications running inside a container to be accessed from the host machine by mapping a host port to a container port. Docker provides commands such as docker ps to view running containers, docker inspect to examine detailed container configuration and network settings, docker stop to gracefully stop a container, and docker rm to remove unused containers. These commands help efficiently manage the lifecycle of Docker containers and verify their network configuration and status. Commands Used • Created and started the Docker container in detached mode with port mapping: docker run -d -p 8081:5000 --name flask-network-demo my-flask-app • Listed all running Docker containers: docker ps • Stopped the running Docker container: docker stop flask-network-demo • Listed all Docker containers, including stopped containers: docker ps -a • Inspected the container's network configuration and settings: docker inspect flask-network-demo • Removed the stopped Docker container: docker rm flask-network-demo

	• Verified that the container had been removed: docker ps -a
Implementation:	**Step 1: Run a Container with Port Mapping** Started Docker Container — Started the Docker container named flask-network-demo in detached mode and mapped host port 8081 to container port 5000, allowing the application to run in the background. **Step 2: Access the Application via Your Browser** Verified Application Access — Accessed the Dockerized Flask application through http://localhost:8081 and verified that the application was successfully running inside the Docker container. **Step 3: List Running Containers** Listed Running Containers — Displayed all currently running Docker containers using the docker ps command and verified that the flask-network-demo container was running with port mapping from 8081 on the host to 5000 inside the container. **Step 4: Stop the Running Container** Stopped Docker Container — Stopped the running flask-network-demo container using the docker stop command, terminating the application gracefully. **Step 5: List Running and All Containers** Verified Container Status — Used the docker ps -a command to display all Docker containers and verified that the flask-network-demo container had stopped successfully and was in the Exited state. **Step 6: Inspect the Container's Network Settings** Inspected Container Network Settings — Used the docker inspect command to view detailed information about the flask-network-demo container, including its NetworkSettings and network configuration. **Step 7: Remove the Stopped Container** Removed Docker Container — Removed the flask-network-demo container using the docker rm command and verified its successful removal by confirming that it no longer appeared in the list of Docker containers.





```
PS C:\Users\Adarsh Jha\Desktop\DevOps-Lab-Assignments\Project-Distributed-Jenkins\portfolio> docker run -d --name agent1 --network jenkins-network jenkins/inbound-agent -url http://jenkins:8080 -secret dbad731d706cfbf7ba4b058d3aa39cdb1ec5b6df59cfa5d9ed6b9deaf0dd4dca -name agent-1 -webSocket -workDir /home/jenkins/agent
8e831cfd2dc7601c6b54d2fdbecaf3fcde4f84e3b012c5ebdd5d3d4732510328
PS C:\Users\Adarsh Jha\Desktop\DevOps-Lab-Assignments\Project-Distributed-Jenkins\portfolio> docker logs agent1
Jul 28, 2026 8:02:03 AM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
INFO: Using /home/jenkins/agent/remoting as a remoting work directory
Jul 28, 2026 8:02:03 AM org.jenkinsci.remoting.engine.WorkDirManager setupLogging
INFO: Both error and output logs will be printed to /home/jenkins/agent/remoting
Jul 28, 2026 8:02:03 AM hudson.remoting.Launcher createEngine
INFO: Setting up agent: agent-1
Jul 28, 2026 8:02:03 AM hudson.remoting.Engine startEngine
INFO: Using Remoting version: 3384.v60d89463d9e0
Jul 28, 2026 8:02:03 AM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
INFO: Using /home/jenkins/agent/remoting as a remoting work directory
Jul 28, 2026 8:02:04 AM hudson.remoting.Launcher$CuiListener status
INFO: WebSocket connection open
Jul 28, 2026 8:02:04 AM hudson.remoting.Launcher$CuiListener status
INFO: Connected
PS C:\Users\Adarsh Jha\Desktop\DevOps-Lab-Assignments\Project-Distributed-Jenkins\portfolio> docker run -d --name agent2 --network jenkins-network jenkins/inbound-agent -url http://jenkins:8080 -secret 2e7bbaa07fb4aaab09b89ba66c56300d5d698954fb2b7ad3ee3be377c80d0c66 -name agent-2 -webSocket -workDir /home/jenkins/agent
7cd1cbd9b9a120c610c7477d6e080a970e5d650ed1eb043643179185ab20b902d
PS C:\Users\Adarsh Jha\Desktop\DevOps-Lab-Assignments\Project-Distributed-Jenkins\portfolio> docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED              STATUS              PORTS
                                                                                                                NAMES
7cd1cbd9b9a1   jenkins/inbound-agent   "/usr/local/bin/jenk…"   About a minute ago   Up About a minute
                                                                                                                agent2
8e831cfd2dc7   jenkins/inbound-agent   "/usr/local/bin/jenk…"   5 minutes ago        Up 5 minutes
                                                                                                                agent1
cfdd6c610bc2   jenkins/jenkins:lts     "/usr/bin/tini -- /u…"   3 days ago           Up 17 minutes       0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp   jenkins
PS C:\Users\Adarsh Jha\Desktop\DevOps-Lab-Assignments\Project-Distributed-Jenkins\portfolio>
```



```
PS C:\WINDOWS\system32> docker images
IMAGE                                                ID             DISK USAGE   CONTENT SIZE   EXTRA
ad4r5h/flask-app:latest                              a829a9538148       1.62GB          417MB
alpine:latest                                        28b05fe8b56d         13MB         3.93MB
assignment-tw2-advanced-devops-flask-app:latest      7149623d568c       1.62GB          417MB
docker.n8n.io/n8nio/n8n:latest                       cda6bafc7bb4       2.47GB          387MB    U
gcr.io/k8s-minikube/kicbase:v0.0.50                  b97074569ae9       1.94GB          545MB
gcr.io/k8s-minikube/kicbase@sha256:eb4fec00e8ad70adf8e6436f195cc429825ffb85f95afcdb5d8d9deb576f3e93   eb4fec00e8ad       1.94GB          545MB    U
jenkins-agent-maven:latest                           03fcc8aebfb9        821MB          240MB    U
jenkins/inbound-agent:latest                         6274f49f8642        479MB          151MB
jenkins/jenkins:lts                                  f4f05e6cd140        801MB          290MB    U
jenkins:2.60.3                                       eeb4850eb65f       1.08GB          316MB
my-flask-app:latest                                  0d96368293d9        200MB         48.8MB    U
n8nio/n8n:latest                                     cda6bafc7bb4       2.47GB          387MB    U
postgres:15                                          74e110c41804        633MB          164MB
react-docker-app:latest                              8b9560310983        547MB          138MB    U
PS C:\WINDOWS\system32> docker rmi ad4r5h/flask-app:latest
Untagged: ad4r5h/flask-app:latest
Deleted: sha256:a829a9538148b0a0860f08bc0fde5401bfa78a74087342c8955cb518eac54fef
PS C:\WINDOWS\system32> docker pull ad4r5h/flask-app:latest
latest: Pulling from ad4r5h/flask-app
d77197981b22: Pull complete
243f68c26961: Pull complete
8175c191d4e6: Pull complete
ef1cddbdf407: Download complete
Digest: sha256:a829a9538148b0a0860f08bc0fde5401bfa78a74087342c8955cb518eac54fef
Status: Downloaded newer image for ad4r5h/flask-app:latest
docker.io/ad4r5h/flask-app:latest
PS C:\WINDOWS\system32>
```

Subject Faculty Name: Prof. Arijit Dutta

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

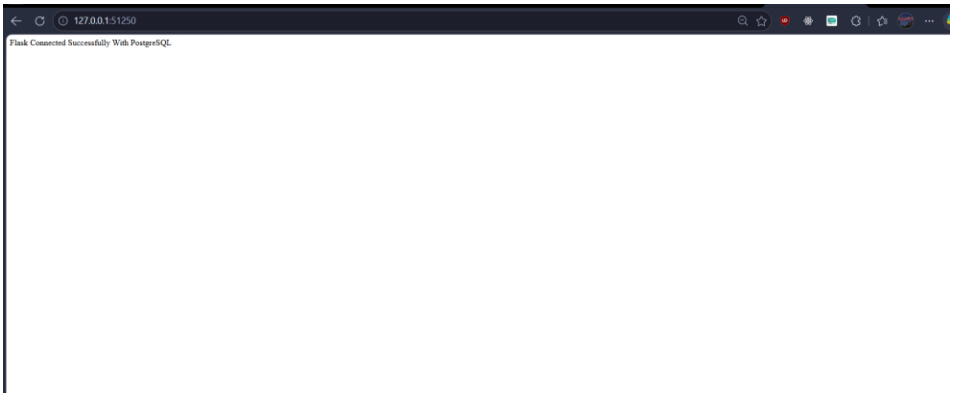
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 10

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Alaa sokhni
PRN No.	23070122276
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	26th October, 2026
Title of Assignment:	Docker Volumes & Data Persistence
Theory:	Docker volumes provide persistent storage for containers by storing data outside the container's writable layer. Unlike container storage, Docker volumes remain intact even after a container is stopped or removed, allowing data to be shared across multiple container instances. By mounting a named volume to a directory inside the container, applications can read and write data that persists independently of the container lifecycle. This makes Docker volumes ideal for storing application data, logs, databases, and other important files that should not be lost when containers are recreated. **Commands Used** • Built the updated Docker image with a new version tag:. • Created a Docker named volume: • Started a container with the named volume mounted: • Stopped the running container: • Removed the stopped container: • Started a new container using the same Docker volume:

Implementation:	**Step 1: Modify the Python Application** Modified Flask Application — Updated the Flask application to write timestamped greetings to a log file inside the /app/data directory and configured the application to run on port **Step 2: Update the Dockerfile** Updated Dockerfile — Modified the Dockerfile to expose , matching the updated Flask application configuration. **Step 3: Build the New Docker Image** Built Updated Docker Image — Built a new Docker image named my-flask-app:v2 containing the modified Flask application configured for Docker volume-based data persistence. **Step 4: Create a Docker Named Volume** Created Docker Volume — Created a Docker named volume called my-app-data to store application data independently of the container, ensuring data persistence across container restarts and removal. **Step 5: Access the Application and Write Data** Verified Data Logging — Accessed the Flask application and refreshed the page multiple times to verify that timestamped greetings were successfully written to the persistent log file stored in the Docker volume. **Step 6: Stop and Remove the Container** Stopped and Removed Container — Stopped the flask-volume-demo container and removed it using the docker stop and docker rm commands while preserving the application data stored in the Docker named volume. **Step 7: Verify Data Persistence** Verified Data Persistence — Accessed the Flask application after starting a new container with the same Docker named volume and confirmed that the previously stored log entries were retained. This demonstrated successful data persistence across multiple container instances. 

```
PS C:\WINDOWS\system32> docker images
                                                
```

Subject Faculty Name: Prof. Arijit Dutta

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 11

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Alaa sokhni
PRN No.	23070122276
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	26th October, 2026
Title of Assignment:	Docker Compose for Multi-Container App
Theory:	Docker Compose is a tool used to define and manage multi-container Docker applications using a single docker-compose.yml file. It allows multiple services, such as a web application and a database, to be configured, started, and managed together with a single command. Docker Compose automatically creates a network for communication between services, manages named volumes for persistent storage, and simplifies deployment by eliminating the need to start each container individually. This makes it easier to develop, test, and maintain applications that consist of multiple interconnected containers. **Commands Used** • Created a new project directory and navigated into it: mkdir my-multi-app cd my-multi-app • Started the multi-container application in detached mode: docker compose up -d • Verified the running containers: docker compose ps *(or docker ps if docker compose ps was not available)* • Stopped and removed the containers and network created by Docker Compose: docker compose down

Step 1: Create the Project Directory

Created Project Directory — Created a new project directory named my-multi-app and navigated into it to store the Docker Compose configuration.

Step 2: Create docker-compose.yml

Created Docker Compose Configuration — Created and saved the docker-compose.yml file defining the Flask web service, Redis service, Docker volumes, and networking configuration for the multi-container application.

Step 3: Start the Multi-Container Application

Started Multi-Container Application — Started the multi-container application using Docker Compose, which created the required network, volumes, and launched both the Flask web service and POSTGRES service.

Step 4: Access the Web Application

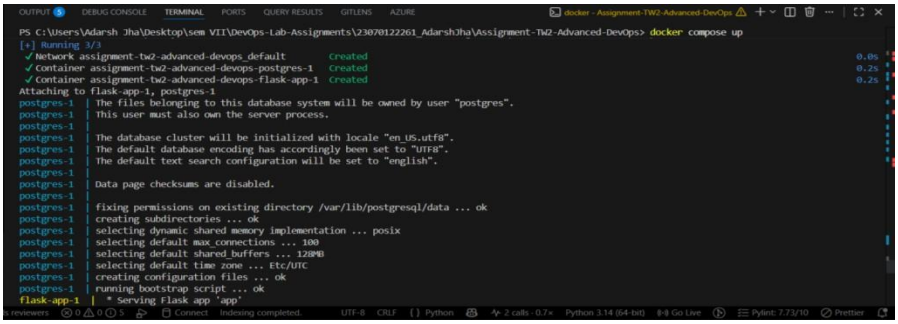
Verified Web Application — Accessed the Flask application through and verified that the web service was successfully running as part of the Docker Compose application.

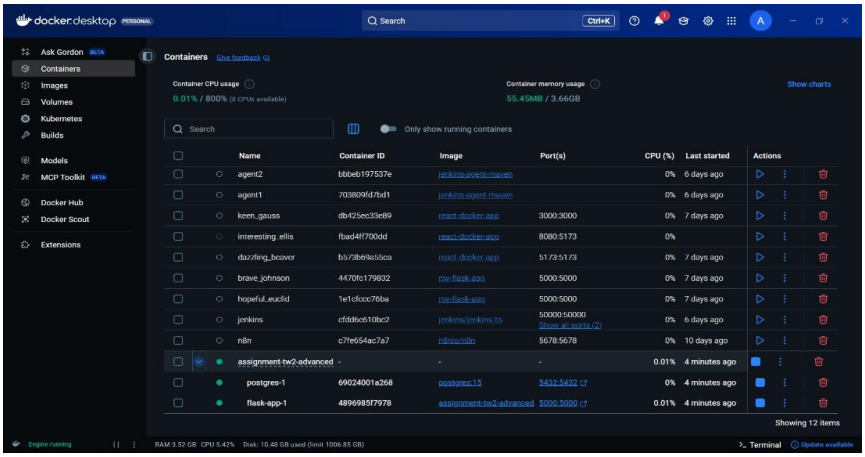
Step 5: Verify the Running Services

Verified Running Services — Verified that the Docker Compose application was running successfully by listing the active containers, including the Flask web service and the postgre service.

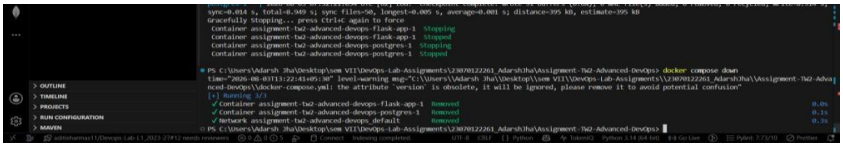
Step 6: Stop and Remove the Multi-Container Application

Stopped Multi-Container Application — Stopped and removed the Docker Compose application, including the running containers and the automatically created network, using the docker compose down command.









Subject Faculty Name: Prof. Arijit Dutta

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

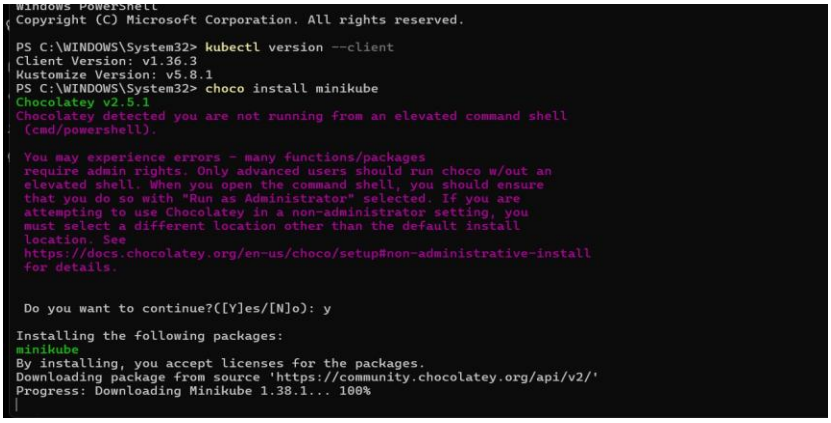
(Established under section 3 of the UGC Act, 1956)

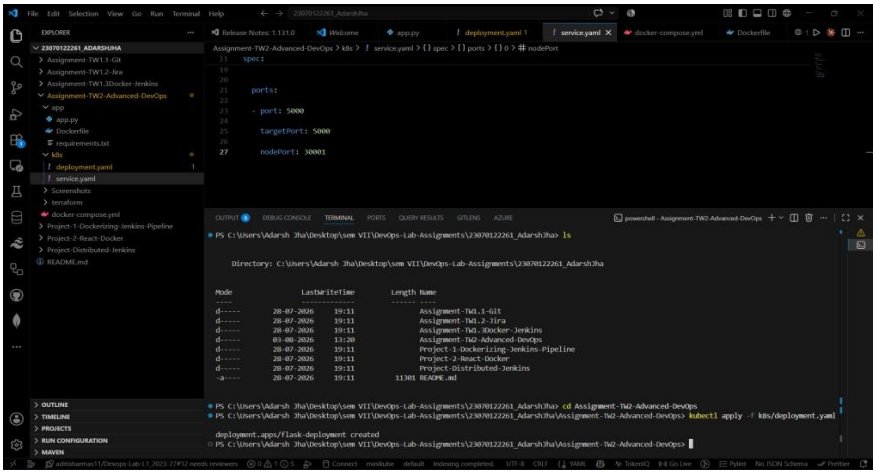
Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

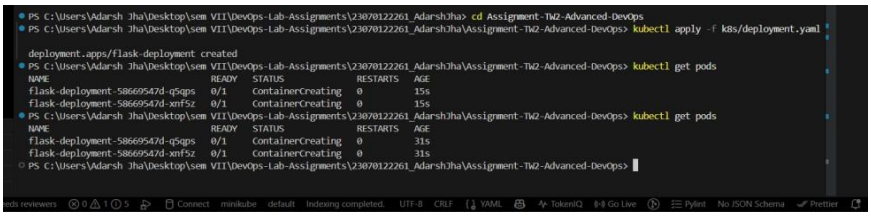
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 12

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Alaa sokhni
PRN No.	23070122276
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	26th October, 2026
Title of Assignment:	Kubernetes Pod & Deployment
Theory:	Kubernetes is an open-source container orchestration platform used to automate the deployment, scaling, and management of containerized applications. A Pod is the smallest deployable unit in Kubernetes and contains one or more containers that share the same network and storage resources. A Deployment manages Pods by maintaining the desired number of replicas, automatically replacing failed Pods, and supporting updates without downtime. Kubernetes simplifies application management by ensuring high availability, scalability, and efficient resource utilization across a cluster. **Commands Used** • Checked the Kubernetes client version: kubectl version --client • Verified the Kubernetes cluster information: kubectl cluster-info • Displayed the available cluster nodes: kubectl get nodes • Created the Kubernetes Deployment from the YAML manifest: kubectl apply -f web-deployment.yaml • Listed the running Pods: kubectl get pods • Deleted the Kubernetes Deployment and its Pods: kubectl delete -f web-deployment.yaml

Implementation:	**Step 1: Enable Kubernetes in Docker Desktop** Enabled Kubernetes — Enabled Kubernetes in Docker Desktop and verified that the local Kubernetes cluster was successfully initialized. **Step 2: Verify kubectl** Verified Kubernetes Cluster — Verified that the local Kubernetes cluster was running successfully using kubectl cluster-info and confirmed that the control-plane node was in the Ready state using kubectl get nodes. **Step 3: Create the Kubernetes Deployment YAML** Created Kubernetes Deployment Manifest — Created the web-deployment.yaml file inside the k8s-manifests directory, defining a Kubernetes Deployment with two replicas of the Flask web application using the Docker image my-flask-app:v2. **Step 4: Apply the Deployment Manifest** Applied Deployment Manifest — Applied the Kubernetes Deployment manifest using kubectl apply, creating the Deployment resource in the local Kubernetes cluster. **Step 5: Verify the Pods** Verified Running Pods — Verified that the Kubernetes Deployment successfully created two running Pods, each hosting an instance of the Flask web application. **Step 6: Clean Up** Deleted Kubernetes Deployment — Removed the Kubernetes Deployment and the Pods it created using the kubectl delete command, cleaning up the local Kubernetes cluster. 





Subject Faculty Name: Prof. Arijit Dutta

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

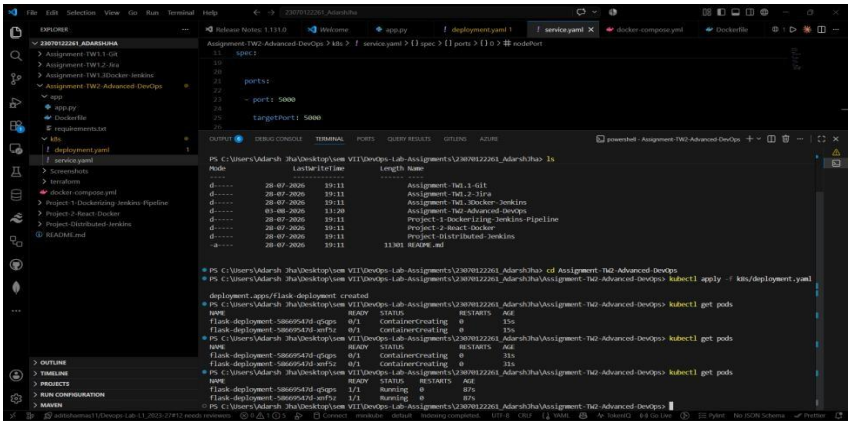
Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

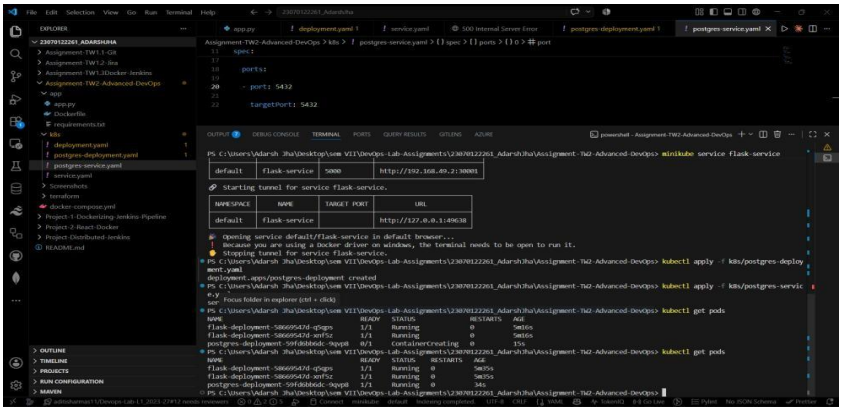
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

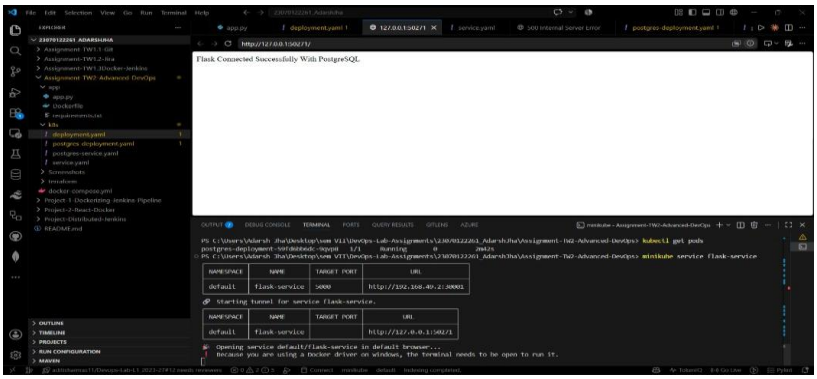
Assignment No. 13

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Alaa sokhni
PRN No.	23070122276
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	26th October, 2026
Title of Assignment:	Kubernetes Service & Networking
Theory:	A Kubernetes Service is an abstraction that provides a stable network endpoint for accessing one or more Pods in a Kubernetes cluster. Since Pods are temporary and their IP addresses may change, a Service uses labels and selectors to route incoming traffic to the appropriate Pods. A NodePort Service exposes the application outside the cluster by allocating a port on each cluster node, allowing external access to the application. During local development, kubectl port-forward can also be used to securely forward traffic from a local machine to a Kubernetes Service for testing and verification. This approach simplifies communication between applications and ensures reliable access to containerized workloads. **Commands Used** • Created the Kubernetes Deployment: kubectl apply -f web-deployment.yaml • Verified that the Pods were running: kubectl get pods • Created the Kubernetes Service: kubectl apply -f web-service.yaml • Displayed the available Kubernetes Services: kubectl get service • Displayed detailed information about the Service: kubectl describe service web-app-service • Displayed the Service endpoints: kubectl get endpoints

	• Displayed the running Pods with labels: kubectl get pods --show-labels • Displayed the application logs: kubectl logs web-app-deployment-<pod-name> • Forwarded the Kubernetes Service to the local machine: kubectl port-forward service/web-app-service 8085:80 • Deleted the Kubernetes Service: kubectl delete -f web-service.yaml • Deleted the Kubernetes Deployment: kubectl delete -f web-deployment.yaml
Implementation:	**Step 1: Create the Kubernetes Service YAML** Created Kubernetes Service Manifest — Created the web-service.yaml file defining a Kubernetes NodePort Service to expose the Flask web application running in the Kubernetes Pods. **Step 2: Apply the Servic** Applied Kubernetes Service — Applied the Kubernetes Service manifest using kubectl apply, creating a NodePort Service for the deployed Flask application. **Step 3: Find the Service Information** Verified Kubernetes Service — Verified the Kubernetes NodePort Service and identified the assigned NodePort used to expose the Flask web application outside the Kubernetes cluster. **Step 4: Access the Application** Accessed Application Through Kubernetes Service — Used kubectl port-forward to forward the Kubernetes Service to the local machine and successfully accessed the Flask web application at , verifying that it was running correctly. **Step 5: Clean Up** Deleted Kubernetes Service — Removed the Kubernetes Service using the kubectl delete command, cleaning up the networking resources created for the application.







Subject Faculty Name: Prof. Arijit Dutta

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 14

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Alaa sokhni
PRN No.	23070122276
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	26th October, 2026
Title of Assignment:	CI/CD: Jenkins Pipeline building Docker Image
Theory:	A Jenkins Pipeline automates the Continuous Integration and Continuous Deployment (CI/CD) process by executing a series of predefined stages such as source code checkout, build, test, and deployment. In this experiment, a Docker image was built automatically using Jenkins after fetching the application source code from a GitHub repository. The docker build command created a versioned Docker image using the Jenkins build number as the image tag, ensuring unique image versions for every pipeline execution. This integration of Jenkins and Docker enables consistent, repeatable, and automated application builds, reducing manual effort and improving software delivery efficiency. **Commands Used** • docker build -t my-jenkins-docker . • docker run -d --name myjenkins --restart unless-stopped -u root -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home -v /var/run/docker.sock:/var/run/docker.sock my-jenkins-docker • docker exec -it myjenkins docker --version • docker exec -it myjenkins rm -rf /var/jenkins_home/workspace/Declarative-Pipeline-Demo • git add . • git commit -m "Add Dockerfile for Jenkins pipeline" • git push origin main • docker images

Implementation:

Step 1: Access and Edit Your Pipeline Configuration

Accessed Jenkins Pipeline Configuration — Opened the existing Declarative Jenkins Pipeline and navigated to the Pipeline configuration section to modify the pipeline script.

Step 2: Add the Build Docker Image Stage

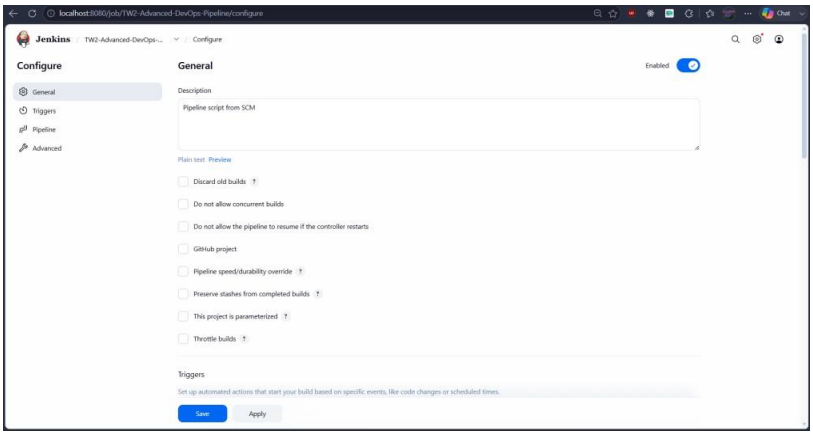
Added Docker Build Stage — Modified the existing Declarative Jenkins Pipeline by adding a new stage to build the Docker image using the docker build command with the Jenkins build number as the image tag.

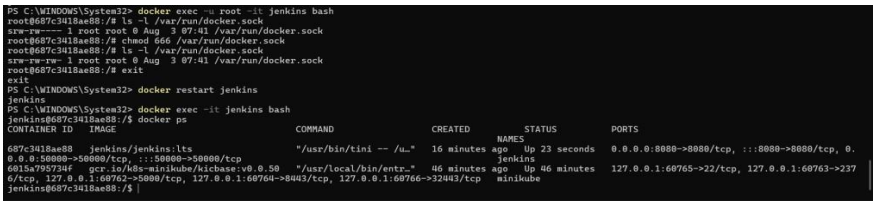
Step 3: Run the Pipeline

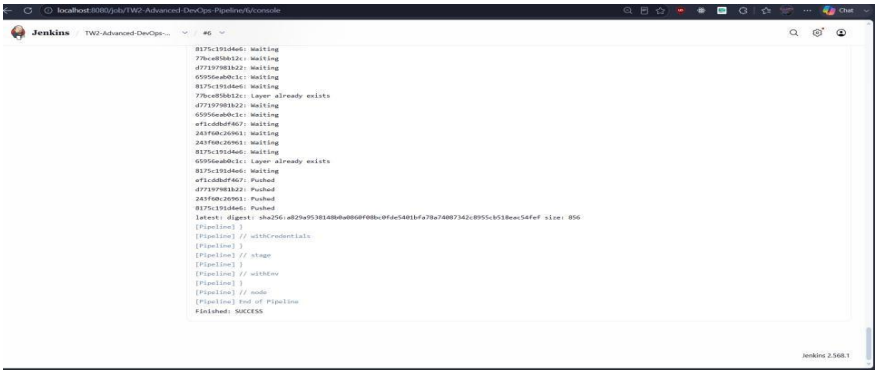
Executed Jenkins Pipeline — Successfully executed the Jenkins Declarative Pipeline containing the Docker image build stage. The pipeline completed successfully, and the Docker image was built and tagged without errors.

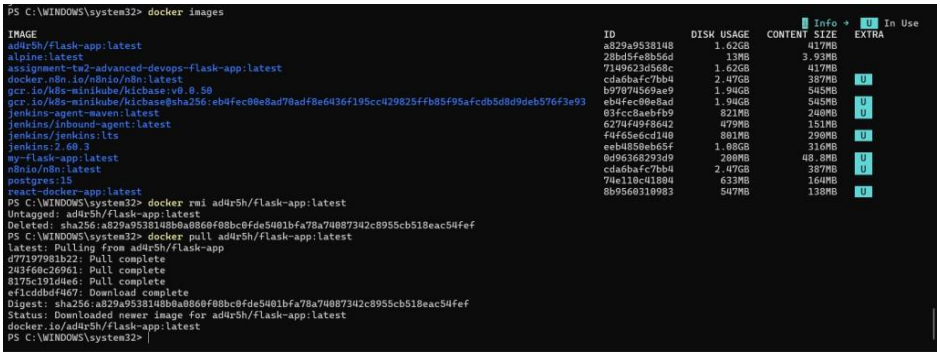
Step 4: Verify the Docker Image

Verified Docker Image — Verified that the Docker image (my-flask-app:8) was successfully built by the Jenkins Pipeline and stored in the local Docker image repository using the docker images command.









Subject Faculty Name: Prof. Arijit Dutta

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 15

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Alaa sokhni
PRN No.	23070122276
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	26th October, 2026
Title of Assignment:	Basic Infrastructure as Code (IaC) with Terraform
Theory:	Infrastructure as Code (IaC) is the practice of managing and provisioning infrastructure using configuration files instead of manual processes. Terraform is an open-source IaC tool developed by HashiCorp that enables users to define, provision, modify, and destroy infrastructure resources in a consistent and automated manner. In this experiment, Terraform was used with the Local provider to create, update, and destroy a local text file. The commands terraform init, terraform plan, terraform apply, and terraform destroy demonstrated Terraform's workflow of initializing the working directory, previewing infrastructure changes, applying the desired configuration, and safely removing managed resources. This approach ensures repeatable deployments, version control of infrastructure, and simplified infrastructure management. **Commands Used** • terraform --version – Checks whether Terraform is installed correctly and displays the installed version. • mkdir my-terraform-iac – Creates a new directory for storing the Terraform project files. • cd my-terraform-iac – Navigates to the Terraform project directory. • terraform init – Initializes the Terraform working directory by downloading the required providers and preparing the project. • terraform plan – Generates an execution plan to preview the infrastructure changes before applying them. • terraform apply – Applies the Terraform configuration and creates or

	updates the resources defined in the configuration file. • terraform destroy – Removes all resources managed by the current Terraform configuration.
Implementation:	**Step 1: Install Terraform** Installed Terraform — Successfully installed Terraform on the Windows system and verified the installation using the terraform --version command. **Step 2: Create the Project Directory** Created Terraform Project Directory — Created a new project directory named my-terraform-iac and navigated into it to store the Terraform configuration files. **Step 3: Create the Terraform Configuration File (main.tf)** Created Terraform Configuration File — Created the main.tf configuration file and defined the local provider along with a local_file resource to create a text file using Terraform. **Step 4: Initialize Terraform** Initialized Terraform — Initialized the Terraform working directory using the terraform init command, which downloaded the required provider plugins and prepared the project for execution. **Step 5: Plan the Changes** Generated Terraform Execution Plan — Generated an execution plan using the terraform plan command to preview the changes Terraform would make. The plan indicated that one new resource (local_file.my_example_file) would be created with no modifications or deletions. **Step 6: Apply the Changes** Applied Terraform Configuration — Applied the Terraform configuration using the terraform apply command, which successfully created the example.txt file as defined in the main.tf configuration. The generated file was verified in the project directory. **Step 7: Modify the Content and Re-apply** Command: terraform plan Generated Updated Execution Plan — Modified the main.tf configuration and generated an updated execution plan using the terraform plan command. Terraform detected that the existing local_file resource would be replaced to apply the updated content. Command: terraform apply

Applied Updated Terraform Configuration — Applied the modified Terraform configuration using the terraform apply command. Terraform successfully recreated the local_file resource with the updated content.

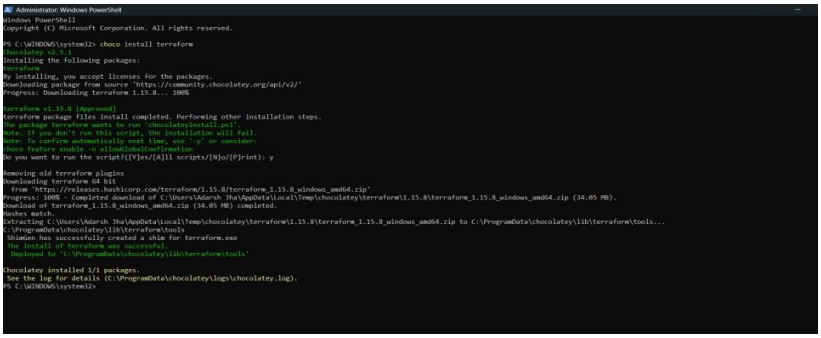
Verified Updated File Content — Verified that the example.txt file was successfully updated with the new content after applying the modified Terraform configuration.

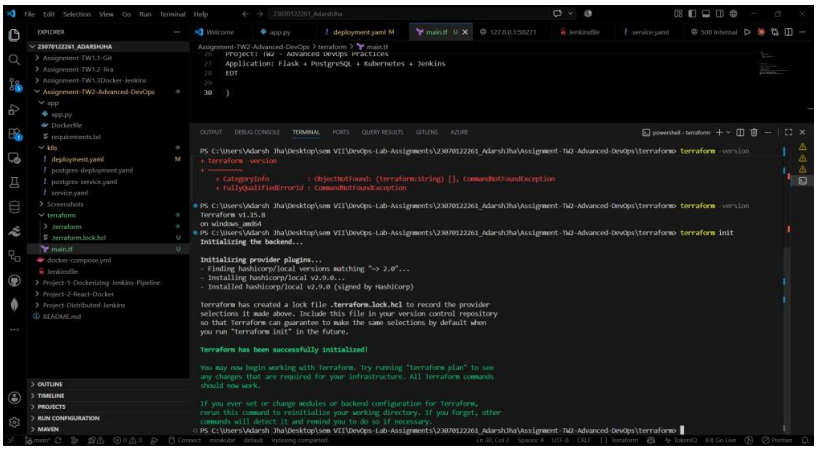
Step 8: Destroy the Resource

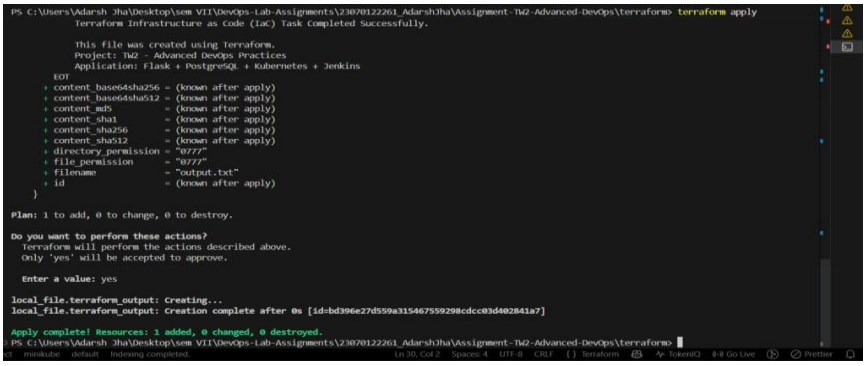
Command: terraform destroy

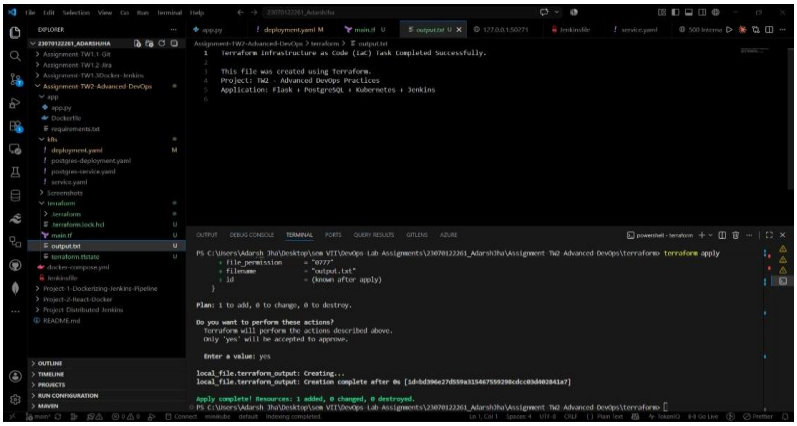
Destroyed Terraform Resource — Executed the terraform destroy command, confirmed the destruction by entering yes, and successfully removed the Terraform-managed local_file resource from the local system.

Verified Resource Deletion — Verified that the example.txt file was successfully removed from the project directory after the Terraform-managed resource was destroyed.









```hcl
terraform {

  required_providers {

    local = {
      source  = "hashicorp/local"
      version = "~> 2.0"
    }

  }

}


provider "local" {}


resource "local_file" "terraform_output" {

  filename = "output.txt"

  content = <<EOT
Terraform Infrastructure as Code (IaC) Task Completed Successfully.

This file was created using Terraform.
Project: TW2 - Advanced DevOps Practices
Application: Flask + PostgreSQL + Kubernetes + Jenkins
EOT

}
```

Subject Faculty Name: Prof. Arijit Dutta